

TÀI LIỆU KIẾN TRÚC HỆ THỐNG

KERNEL VIGIL

ĐẶC TẢ YÊU CẦU PHẦN MỀM (SRS) & THIẾT KẾ KIẾN TRÚC (SDD)
HỆ THỐNG GIÁM SÁT THI CỬ BẢO MẬT CẤP NHÂN (RING 0)

Thành viên phụ trách	Nguyễn Đức Huy (Ring 0 - Core Driver & System Architect) Cộng sự Team (Ring 3 - User App & Network Logic)
Phiên bản tài liệu	v1.2.0 (Bản bổ sung thiết kế đồng bộ, quản lý IRQL & cơ chế tự vệ toàn diện)
Đơn vị áp dụng nghiên cứu	Ngành Công nghệ Thông tin - CMC University
Ngày phát hành	05 tháng 06, 2026

THUẬT NGỮ CỐT LÕI (GLOSSARY)

Thuật ngữ	Mô tả kỹ thuật hành vi
Ring 0 (Kernel Mode)	Mức đặc quyền cao nhất của CPU, cho phép driver thực thi các chỉ thị phần cứng trực tiếp và có quyền truy cập không giới hạn vào bộ nhớ toàn cục.
Ring 3 (User Mode)	Không gian hạn chế đặc quyền dành cho ứng dụng người dùng, biệt lập với tài nguyên hệ thống lõi ngoại trừ qua các API cổng kiểm soát.
IRQL (Interrupt Request Level)	Mức ưu tiên ngắt phần cứng/phần mềm trong Windows Kernel. Quản lý IRQL sai cách (ví dụ: truy cập bộ nhớ paged tại DISPATCH_LEVEL) gây BSOD lập tức.
Inverted Call Mechanism	Cơ chế gọi ngược đảo chiều: Ring 3 gửi IRP chờ sẵn dưới Driver; khi Ring 0 phát hiện vi phạm, nó sẽ hoàn thành IRP đó để chủ động đẩy dữ liệu lên Ring 3 ngay lập tức thay vì đợi Ring 3 poll dữ liệu.
BYOVD Attack	Tấn công lạm dụng driver hợp pháp có lỗ hổng (Bring Your Own Vulnerable Driver) để bypass cơ chế bảo vệ của Windows Driver Signature và tắt Kernel Vigil.

1. Tổng quan dự án (Project Overview)

1.1 Giới thiệu bài toán

Trong môi trường giáo dục số hóa hiện nay, các phần mềm thi cử trực tuyến cấp User Mode (nhux Safe Exam Browser - SEB) đang bộc lộ những lỗ hổng bảo mật mang tính kiến trúc. Do hoạt động hoàn toàn ở Ring 3, các hệ thống này dễ dàng bị vượt qua bởi các kỹ thuật can thiệp hệ thống cơ bản như cấu hình

Image File Execution Options (IFEO), kỹ thuật API Hooking nâng cao, hoặc các driver ảo hóa phần cứng điều khiển từ xa.

Dự án Kernel Vigil được ra đời nhằm tạo ra một bước đột phá trong công nghệ giám sát thi cử. Bằng cách dịch chuyển trọng tâm bảo mật xuống tầng Nhân (Ring 0), hệ thống thiết lập quyền kiểm soát tuyệt đối trên thiết bị phần cứng, vô hiệu hóa tận gốc mọi hành vi gian lận kỹ thuật số tinh vi nhất, mang lại sự công bằng triệt để cho người học và sự an tâm tuyệt đối cho các cơ sở đào tạo.

1.2 Mục tiêu kiến trúc

- Kiểm soát độc quyền tầng thấp:** Chặn đứng mọi nỗ lực can thiệp tiến trình, chỉnh sửa Registry, hoặc đánh kèm trình gỡ lỗi (Debugger).
- Xây dựng khung xương sống (Backbone) vững chắc:** Thiết lập một kênh giao tiếp bất khả xâm phạm và có độ ổn định tối đa giữa Ring 0 Driver và Ring 3 Controller.
- Đảm bảo tính toàn vẹn tuyệt đối:** Nếu thành phần bảo mật bị tắt hoặc hỏng hóc, toàn bộ hệ thống thi lập tức rơi vào trạng thái cô lập an toàn (Fail-Safe).
- Vô hiệu hóa môi trường ảo hóa (Anti-Hypervisor/VM):** Chủ động rà soát và từ chối cung cấp môi trường thi nếu phát hiện hệ điều hành đang chạy bên trong máy ảo (VMware, VirtualBox) hoặc bị thao túng bởi một Hypervisor tùy chỉnh ở cấp độ Ring -1.

2. Đặc tả yêu cầu phần mềm (SRS)

2.1 Yêu cầu chức năng (Functional Requirements)

Mã Yêu Cầu	Tên Chức Năng	Mô Tả Chi Tiết Hoạt Động Của Hệ Thống
REQ-F-01	Khởi tạo môi trường thi (Secure Handshake)	Khi Client Ring 3 khởi chạy, nó phải gửi mã IOCTL kích hoạt hợp lệ xuống Driver Ring 0. Driver tiến hành đăng ký các hàm callback hệ thống và khóa các tài nguyên quan trọng.
REQ-F-02	Giám sát tiến trình (Process Monitoring)	Sử dụng cơ chế đăng ký callback cấp nhân để bắt tất cả sự kiện khởi chạy tiến trình hệ thống. Chặn đứng ngay lập tức các tiến trình lạ không nằm trong danh sách trắng (Whitelist).
REQ-F-03	Bảo vệ Registry chủ động	Chặn mọi hành vi can thiệp vào các khóa registry nhạy cảm, đặc biệt là cấu hình Image File Execution Options (IFEO) để tránh lỗi bypass qua cơ chế Debugger Injection.
REQ-F-04	Chống can thiệp bộ nhớ (Anti-Tampering)	Ngăn cấm các tiến trình ở Ring 3 (kể cả quyền Administrator) mở Handle với quyền truy cập sâu (PROCESS_ALL_ACCESS) vào không gian bộ nhớ của phần mềm thi.
REQ-F-05	Cô lập không gian làm việc (Secure Desktop)	Tạo và chuyển đổi hệ thống sang một Desktop ảo biệt lập hoàn toàn, vô hiệu hóa các phím tắt can thiệp hệ thống như Windows, Alt+Tab, Task Manager.

Mã Yêu Cầu	Tên Chức Năng	Mô Tả Chi Tiết Hoạt Động Của Hệ Thống
REQ-F-06	Chống thao tác đối tượng lỗi trực tiếp (Anti-DKOM)	Phát hiện các phần mềm gian lận sử dụng kỹ thuật ẩn tiến trình (Hidden Processes) bằng cách đối chiếu chéo danh sách luồng thực thi cấp nhân (EPROCESS block) so với danh sách trả về từ Windows API.
REQ-F-07	Bảo vệ Page Tables & CR3	Giám sát thanh ghi CR3 và tính toàn vẹn của Page Table Entries (PTE) để ngăn chặn các kỹ thuật re-mapping bộ nhớ vật lý nhằm vượt qua các chốt chặn đọc/ghi.
REQ-F-08	Thông báo vi phạm chủ động (Asynchronous Alert)	[BỔ SUNG] Khi Ring 0 chặn tiến trình/Registry vi phạm, hệ thống phải kích hoạt cơ chế báo động thời gian thực truyền thẳng về Ring 3 lập tức mà không chờ chu kỳ Heartbeat của ứng dụng.

2.2 Yêu cầu phi chức năng (Non-Functional Requirements)

- **Độ ổn định hệ thống (Zero BSOD Policy):** Do mã nguồn chạy ở Ring 0, mọi cấu trúc giải phóng bộ nhớ, bắt ngoại lệ hệ thống (Structured Exception Handling - SEH) phải được kiểm soát tuyệt đối, đảm bảo tỉ lệ gây sập màn hình xanh dưới 0.01%.
- **Hiệu năng tối ưu (Low Overhead):** Các hàm callback kiểm tra tiến trình và registry phải có độ trễ thực thi nhỏ hơn 1ms để tránh gây giật lag hệ điều hành của thí sinh.
- **Bảo mật kênh truyền (IPC Security & Input Validation):** Kênh giao tiếp I/O Control giữa ứng dụng và driver phải được xác thực bằng mã định danh phiên độc quyền, chống lại các cuộc tấn công replay từ phần mềm giả lập ngoại vi. Đặc biệt, hệ thống phải thực hiện kiểm tra giới hạn bộ đệm (Buffer Bounds Checking) cực kỳ khắt khe để loại bỏ nguy cơ tràn bộ đệm tại Ring 0.
- **An toàn luồng xử lý (Concurrency and Synchronization Security):** **[BỔ SUNG]** Đảm bảo cơ chế khóa (Locking mechanism) không gây tình trạng deadlock giữa luồng xử lý callback hệ thống và luồng giao tiếp IOCTL.

3. Thiết kế kiến trúc hệ thống (SDD)

3.1 Mô hình phân tầng hệ thống (Layered Architecture)

Hệ thống được chia làm 3 lớp kiến trúc rõ rệt, đảm bảo nguyên tắc phân rã trách nhiệm (Separation of Concerns):

- **Tầng Nhân (Ring 0) - Do Nguyễn Đức Huy phụ trách:** Thực thi các lệnh đặc quyền cao nhất, áp đặt cấu hình bảo mật trực tiếp lên CPU, RAM và hệ thống tệp tin/registry thông qua Windows Kernel API.
- **Tầng Giao Tiếp (The Bridge) - Phối hợp phát triển:** Sử dụng cơ chế I/O Control Packets (IRP) và bộ đệm bộ nhớ chia sẻ (Shared Memory Area) để chuyển đổi dữ liệu an toàn xuyên qua ranh giới giữa Kernel và User Mode.

- **Tầng Ứng Dụng (Ring 3) - Do Cộng sự phụ trách:** Xây dựng giao diện thi bằng C++/Qt hoặc C#, quản lý luồng đề thi qua Webview2 Core, giám sát webcam thông qua AI local và đồng bộ dữ liệu mã hóa về máy chủ nhà trường.

3.2 Chi tiết Module cấp Nhân (Ring 0 Core Driver)

3.2.1 Hàm khởi tạo DriverEntry

Khi hệ thống nạp Driver, hàm này sẽ thực hiện:

- Tạo đối tượng thiết bị (IoCreateDevice) với tên định danh cục bộ `\Device\KernelVigil`.
- Tạo liên kết tượng trưng (IoCreateSymbolicLink) tới `\DosDevices\KernelVigil` để Ring 3 có thể mở handle kết nối qua hàm CreateFile.
- Đăng ký các điểm phân phối lệnh (Dispatch Routines) cho các tác vụ: `IRP_MJ_CREATE`, `IRP_MJ_CLOSE`, và quan trọng nhất là `IRP_MJ_DEVICE_CONTROL`.

3.2.2 Cơ chế thực thi bảo mật chủ động (Enforcement Callbacks)

- **Giám sát Registry:** Sử dụng hàm `CmRegisterCallbackEx`. Khi có bất kỳ tiến trình nào cố gắng thay đổi khóa Image File Execution Options, Driver sẽ kiểm tra chữ ký nếu không thuộc hệ thống sẽ trả về trạng thái `STATUS_ACCESS_DENIED` trực tiếp, bẻ gãy cuộc tấn công IFEO.
- **Giám sát Tiến trình:** Sử dụng hàm `PsSetCreateProcessNotifyRoutineEx`. Mỗi khi một tệp tin thực thi (.exe) chuẩn bị được nạp vào bộ nhớ, CPU sẽ nhảy vào hàm filter của Kernel Vigil trước. Nếu định danh tiến trình nằm trong danh sách đen (Blacklist), trường `CreateInfo->CreationStatus` sẽ bị gán bằng mã lỗi, ngăn chặn tiến trình khởi tạo hoàn toàn.
- **Xác thực giao tiếp IOCTL (Input Validation):** Thiết lập cơ chế kiểm tra kích thước bộ đệm (Buffer Bounds Checking) nghiêm ngặt tại hàm Dispatch Routine. Mọi gói tin truyền qua cơ chế `METHOD_BUFFERED` đều được xác minh `InputBufferLength` trước khi đọc cấu trúc bộ nhớ, loại trừ hoàn toàn rủi ro tin tặc đẩy payload biến dạng gây lỗi tràn bộ nhớ (Buffer Overflow) dưới kernel.

3.2.3 [BỔ SUNG] Quản lý Đồng bộ hóa Luồng & Ngắt Hệ thống (IRQL Management)

Mã nguồn chạy tại hệ thống Callback của Windows hoạt động ở các mức IRQL khác nhau. Để tránh xung đột tài nguyên giữa luồng đăng ký danh sách Whitelist từ Ring 3 truyền xuống và luồng đọc dữ liệu khi có tiến trình khởi chạy, thiết kế bổ sung hai chốt chặn khóa hạt nhân:

- **Mức Ngắt Thấp (IRQL < DISPATCH_LEVEL - ví dụ: PASSIVE_LEVEL / APC_LEVEL):** Sử dụng cơ chế **Executive Fast Mutex** thông qua cặp hàm **ExAcquireFastMutex** và **ExReleaseFastMutex** để đồng bộ hóa việc chèn/xóa các phần tử trong danh sách Whitelist của Driver. Cơ chế này giảm overhead và cho phép luồng ngủ (sleep) nếu cần tài nguyên.
- **Mức Ngắt Cao (IRQL = DISPATCH_LEVEL):** Đối với các hệ thống lọc nâng cao hoặc thao tác trên hàng đợi yêu cầu bất đồng bộ, Driver buộc phải chuyển sang sử dụng khóa xoay **KSPIN_LOCK** qua hàm **KeAcquireSpinLockAtDpcLevel**. **Lưu ý cực kỳ quan trọng:** Tuyệt đối không truy cập bộ nhớ ảo có thể bị hoán vị (Paged Pool) trong vùng khóa này, toàn bộ cấu trúc Whitelist/Log bắt buộc phải nằm trên **NonPagedPoolNx**.

3.2.4 [BỔ SUNG] Cơ chế Gọi Ngược Đảo Chiều (Inverted Call Mechanism)

Thay vì ứng dụng Ring 3 liên tục tạo luồng ping lấy log (gây lãng phí tài nguyên CPU), Backbone nâng cao thiết lập kiến trúc **Inverted Call**:

1. Khi kỳ thi bắt đầu, ứng dụng Ring 3 gửi một mã IOCTL đặc biệt (ví dụ: ``IOCTL_KV_LISTEN_EVENT``) xuống Driver. Bộ phân phối lệnh (Dispatch Routine) tại Ring 0 sẽ giữ gói tin IRP này ở trạng thái chờ xử lý (Pending via **IoMarkIrpPending**) và đưa nó vào một hàng đợi an toàn (Interlocked Queue).
2. Ngay khi hàm callback ``PsSetCreateProcessNotifyRoutineEx`` phát hiện mã gian lận, Ring 0 sẽ rút gói IRP đang treo ra khỏi hàng đợi, sao chép cấu trúc dữ liệu lỗi vào bộ đệm của IRP và gọi hàm **IoCompleteRequest** với mã trạng thái **STATUS_SUCCESS**.
3. Ứng dụng khách ở Ring 3 nhận được tín hiệu hoàn thành tác vụ không đồng bộ ngay lập tức từ Windows Subsystem và xử lý hành vi khóa bài thi của sinh viên trong chưa đầy vài micro giây.

3.3 Chi tiết Module cấp Ứng dụng (Ring 3 Client)

- **Tạo màn hình bảo mật:** Gọi API **CreateDesktop** để tạo một không gian làm việc sạch hoàn toàn, không chứa thanh Taskbar, không chứa lối tắt ứng dụng nền. Tiếp theo gọi **SwitchDesktop** để đưa thí sinh vào vùng không gian này.
- **Kênh kiểm tra nhịp đập (Heartbeat Thread):** Khởi chạy một tiểu trình chạy ngầm liên tục gửi mã IOCTL ping xuống Driver định kỳ mỗi 1000ms để xác thực sự tồn tại của driver, tránh trường hợp driver bị bypass bằng các kỹ thuật gỡ nạp lậu (Unload exploit).

CẢNH BÁO THIẾT KẾ: CƠ CHẾ TỰ VỆ CỦA DRIVER (DRIVER SELF-DEFENSE)

Một lỗi hỏng nghiêm trọng của các phần mềm giám sát thông thường là việc sinh viên dùng quyền Administrator để kết thúc (Kill Process) ứng dụng Ring 3 hoặc can thiệp ghi đè bộ nhớ của Client thi. Để bảo vệ Backbone, Module Ring 0 do Huy phụ trách phải triển khai thêm hàm callback bảo vệ tiến trình khách:

Sử dụng `ObRegisterCallbacks` đăng ký bộ lọc cho đối tượng `PsProcessType`. Khi bất kỳ tiến trình nào bên ngoài cố gắng gọi API `OpenProcess` với cờ truy cập sâu như `PROCESS_TERMINATE` (hủy tiến trình), `PROCESS_VM_WRITE` (ghi bộ nhớ), hoặc `PROCESS_VM_READ` (đọc bộ nhớ) hướng tới ứng dụng Ring 3, Driver sẽ tự động xóa bỏ các cờ (Strip Access Rights) này trong cấu trúc `OB_PRE_OPERATION_INFORMATION`, trả về quyền tối thiểu, khiến công cụ hack không thể can thiệp vào Client.

4. Thiết kế giao thức & Cấu trúc dữ liệu

4.1 Khai báo tệp tin cấu trúc xương sống SharedDef.h (Cập nhật phiên bản v1.2.0)

```
#ifndef SHARED_DEF_H
#define SHARED_DEF_H

// Định nghĩa mã Device Type (Nằm trong dải an toàn cho Custom Drivers từ 32768-65535)
#define KERNEL_VIGIL_DEVICE_TYPE 0x8000

// Kỹ thuật định nghĩa mã IOCTL chuẩn Microsoft WDK
#define IOCTL_KV_INITIALIZE_EXAM CTL_CODE(KERNEL_VIGIL_DEVICE_TYPE, 0x900, METHOD_BUFFERED, FILE_ANY_ACCESS)
#define IOCTL_KV_TERMINATE_EXAM CTL_CODE(KERNEL_VIGIL_DEVICE_TYPE, 0x901, METHOD_BUFFERED, FILE_ANY_ACCESS)
#define IOCTL_KV_SEND_HEARTBEAT CTL_CODE(KERNEL_VIGIL_DEVICE_TYPE, 0x902, METHOD_BUFFERED, FILE_ANY_ACCESS)
#define IOCTL_KV_ADD_WHITELIST_PID CTL_CODE(KERNEL_VIGIL_DEVICE_TYPE, 0x903, METHOD_BUFFERED, FILE_ANY_ACCESS)

// [BỔ SUNG V1.2.0] IOCTL phục vụ cho cơ chế đảo chiều gọi (Inverted Call)
#define IOCTL_KV_LISTEN_EVENT CTL_CODE(KERNEL_VIGIL_DEVICE_TYPE, 0x904, METHOD_BUFFERED, FILE_ANY_ACCESS)

// Cấu trúc gói tin dữ liệu truyền tải giữa Ring 3 và Ring 0
typedef struct _EXAM_INIT_DATA {
    ULONG ClientProcessId; // PID của phần mềm thi cấp User Mode
    ULONG SecurityLevelFlags; // Các cờ cấu hình mức độ bảo mật chủ động
    WCHAR SessionToken[64]; // Chuỗi khóa bảo mật chống tấn công Replay
} EXAM_INIT_DATA, *PEXAM_INIT_DATA;

typedef struct _MONITOR_LOG_ENTRY {
    ULONG ConfiscatedProcessId; // PID của tiến trình gian lận bị phát hiện và chặn
    WCHAR ImagePath[256]; // Đường dẫn tệp tin thực thi vi phạm nội quy thi
    ULONG ViolationType; // Phân loại lỗi (1: Chặn Registry, 2: Chặn Process, 3: DKOM)
} MONITOR_LOG_ENTRY, *PMONITOR_LOG_ENTRY;
```

5. Lộ trình triển khai, Chính sách Bảo mật & Kế hoạch kiểm thử

5.1 Lộ trình triển khai tích hợp

Dự án được phân bổ thực thi song song để tối ưu hóa thời gian phối hợp nhóm:

Giai Đoạn / Thời Gian	Nhiệm vụ của Huy (Ring 0)	Nhiệm vụ của Cộng sự (Ring 3)
Giai đoạn 1: Thiết lập (Ngày 1 - Ngày 3)	Xây dựng khung Driver cơ bản, tạo Device Object và thiết lập Symbolic Link thành công.	Cấu hình Project nền tảng, thực thi mã mở kết nối CreateFile tới luồng Driver.
Giai đoạn 2: Handshake (Ngày 4 - Ngày 7)	Hoàn thiện hàm Dispatch Routine, bóc tách cấu trúc EXAM_INIT_DATA gửi xuống từ tầng ứng dụng. Cài đặt hàng đợi Inverted Call.	Thực thi hàm gọi DeviceIoControl, truyền tải mã token phiên bản, kích hoạt vòng lặp Overlapped I/O để nghe sự kiện từ Driver.
Giai đoạn 3: Thực thi (Ngày 8 - Ngày 12)	Tích hợp hàm Callback chặn Registry, khởi tạo tiến trình. Thực thi vá lỗi hỏng IFEO và kích hoạt bảo vệ ObRegisterCallbacks.	Phát triển module Secure Desktop độc quyền, nhúng Webview2 Core hiển thị đề thi mẫu. Cài đặt cơ chế bảo vệ UI.
Giai đoạn 4: Kiểm thử (Ngày 13 - Ngày 15)	Thực hiện gỡ lỗi trực tiếp qua WinDbg, rà soát vùng nhớ Paged/Non-paged Pool loại bỏ rò rỉ bộ nhớ. Thực hiện Kernel Fuzzing áp lực cao.	Thực hiện các kịch bản tấn công thử nghiệm từ Ring 3 để đánh giá hiệu năng phòng vệ tự động của hệ thống.

5.2 Kế hoạch kiểm thử Backbone hệ thống (Test Matrix)

- **Kịch bản Test_01 (Giao tiếp cơ bản - Ping/Pong):** Kiểm tra tính thông suốt của luồng I/O đồng bộ. Yêu cầu phản hồi dưới 0.1ms.
- **Kịch bản Test_02 (Xử lý sự cố đột ngột - Fail-Safe):** Giả lập tình huống ứng dụng khách bị crash. Driver bắt được tín hiệu `IRP_MJ_CLOSE` từ luồng handle cuối cùng, tự động gỡ bỏ toàn bộ callback hệ thống an toàn, tránh treo nhân Windows.
- **Kịch bản Test_03 (Phòng vệ chống can thiệp Client):** Dùng tiến trình quản trị cấp cao chạy quyền System (như Process Hacker) để gửi lệnh Kill Process vào Client Ring 3. Kết quả kỳ vọng: `ObRegisterCallbacks` chặn đứng hành vi, quyền truy cập bị strip hoàn toàn, Client an toàn tuyệt đối.
- **Kịch bản Test_04 (Kiểm thử áp lực IOCTL Fuzzing):** Đẩy hàng vạn gói tin IOCTL lỗi cấu trúc vùng đệm xuống kernel để kiểm thử độ vững chãi của khối bọc lỗi SEH. Kết quả kỳ vọng: Driver từ chối an toàn, hệ thống đạt tỷ lệ ổn định lý tưởng.

5.3 [BỔ SUNG] Điều kiện Tiên quyết về Chữ ký Số & Cơ chế Bảo mật Hệ điều hành

Để triển khai thực tế mã nguồn Driver cấp Nhân lên máy tính của thí sinh, kiến trúc phát triển bắt buộc tuân thủ quy chuẩn ký số nghiêm ngặt của Microsoft:

- **Môi trường Phát triển & Kiểm thử nội bộ (Test Lab):** Do driver chưa có chữ ký số thương mại, hệ thống máy kiểm thử phải chạy lệnh cấu hình nói lỏng: `bcdedit /set testsigning on` để Windows chấp nhận thực thi driver ký bằng chứng chỉ tự tạo (Self-signed Certificate).
- **Môi trường Triển khai Thực tế (Production Deployment):** Kể từ Windows 10/11, mọi driver Ring 0 chạy chế độ chính thức bắt buộc phải được ký bằng **Chứng thư số mở rộng (EV Certificate)** được cấp bởi các tổ chức uy tín (như DigiCert, GlobalSign), sau đó nộp lên cổng Microsoft Hardware Dev Center để vượt qua bài kiểm duyệt chất lượng **WHQL (Windows Hardware Lab Kit)** để có chữ ký số chính thức từ Microsoft.
- **Tuân thủ cơ chế HVCI (Hypervisor-Protected Code Integrity):** Mã nguồn driver phải hoàn toàn tương thích với tính năng Core Isolation của Windows. Cấm sử dụng các vùng nhớ thực thi động tùy tiện, mọi vùng cấp phát dữ liệu (như danh sách Whitelist cấu trúc) bắt buộc dùng cờ chống thực thi mã nguồn độc hại `NonPagedPoolNx`.